03-02-2022

PART - 2

* FUNDAMENTALS OF OOPS :

1) CLASS

2) OBJECT

* UNDERSTANDING OF STATIC

1) Static Method

2) Static variable

3) Static initializer.

* UNDERSTANDING OF NON-STATIC :

1) Non-static Method.

2) Non-static variable

3) Non-static initializer

4) Constructor.

* PRINCIPLES OF OOPS :

1) ENCAPSULATION

2) INHERITANCE

3) ABSTRACTION

4) POLYMORPHISM

* PACKAGES

* MODIFIERS

* NON-PRIMITIVE TYPE CASTING

* FINAL

* SUPER

PART-3

* ARRAYS

* OBJECT CLASS

* STRING CLASS

* EXCEPTION & IETS HANDLING.

* WRAPPER CLASSES.

* INTRODUCTION TO THREADS.

* COLLECTION → LIST
→ SET
→ MAP

* INTRODUCTION TO FILE HANDLING.

FUNDAMENTALS OF OOPS

⇒ OBJECT:-

Anything which has existence in real world is known as Object.

Ex: Car, house, etc.,

Every object will have its own properties / data characteristics / states / behaviour

Ex: Employee.

STATES OF EMPLOYEE	BEHAVIOUR OF EMPLOYEE
* EMP ID * EMPNAME * EMPSAL * Employee Designation	* Login() * Logout() * work() * Report()

Ex 2: LAPTOP

STATES OF LAPTOP	BEHAVIOUR OF LAPTOP
* Brand * Model * RAM * hard disk * price	* browsing() * gaming() * Read() * Type()

NOTE:

Since the states & behaviour belong to one particular object, If ? need to store all this things we need one dedicated memory location. But we dont have any predefined datatype which helps to create such container. Hence, we can design our ocon datatype with the help of a class, which can be used as non-primitive data type.

CLASS:

* class is a keyword. It is used to design a user defined non-primitive datatype to store the details of object

* It can also be considered as blueprint of an object

* In a class, we can create members like method (to store behaviour of an object) as well as variable. (to store states of object).

* Every class name can be a non-primitive data t

* The java language also comes up with lot of built-in classes ie, String, System, etc.

Ex: 1) class Employee (non-primitive data type)
{
// design variables to store states;
// design methods to store behaviours;
}

2) class laptop (non-primitive data type)
{
// design variables to store states;
// design methods to store behaviours
}

NOTE :

1) The non-primitive data types will have only default values ie null.

2) They will not be having default size.

3) Since the name of the class itself can be used a non-primitive data type.

4) We can have any member of non-primitive data type in java

▶ MEMBERS OF CLASS:

Anything which is declared within class block is known as Members of the class.

Ex: Methods, Variables, initializers, constructor, etc,

The member of the class can be classified into 2 types:

1) static members of class.

2) Non-static members of the class.

▶ STATIC MEMBERS OF THE CLASS:

Any member of the class which is prefixed with static modifier is known as static member of a class.

In java, we have following static members:

1) Static Method

2) Static variables

3) Static Initializers.

04/02/2022

▶ STATIC METHODS:-

* A method which is prefixed with static modifier is known as Static Methods.

* The static members will have the memory allocated inside Class static Area.

* Since the static members get the memory inside class static area, they are also known as **CLASS MEMBERS.**

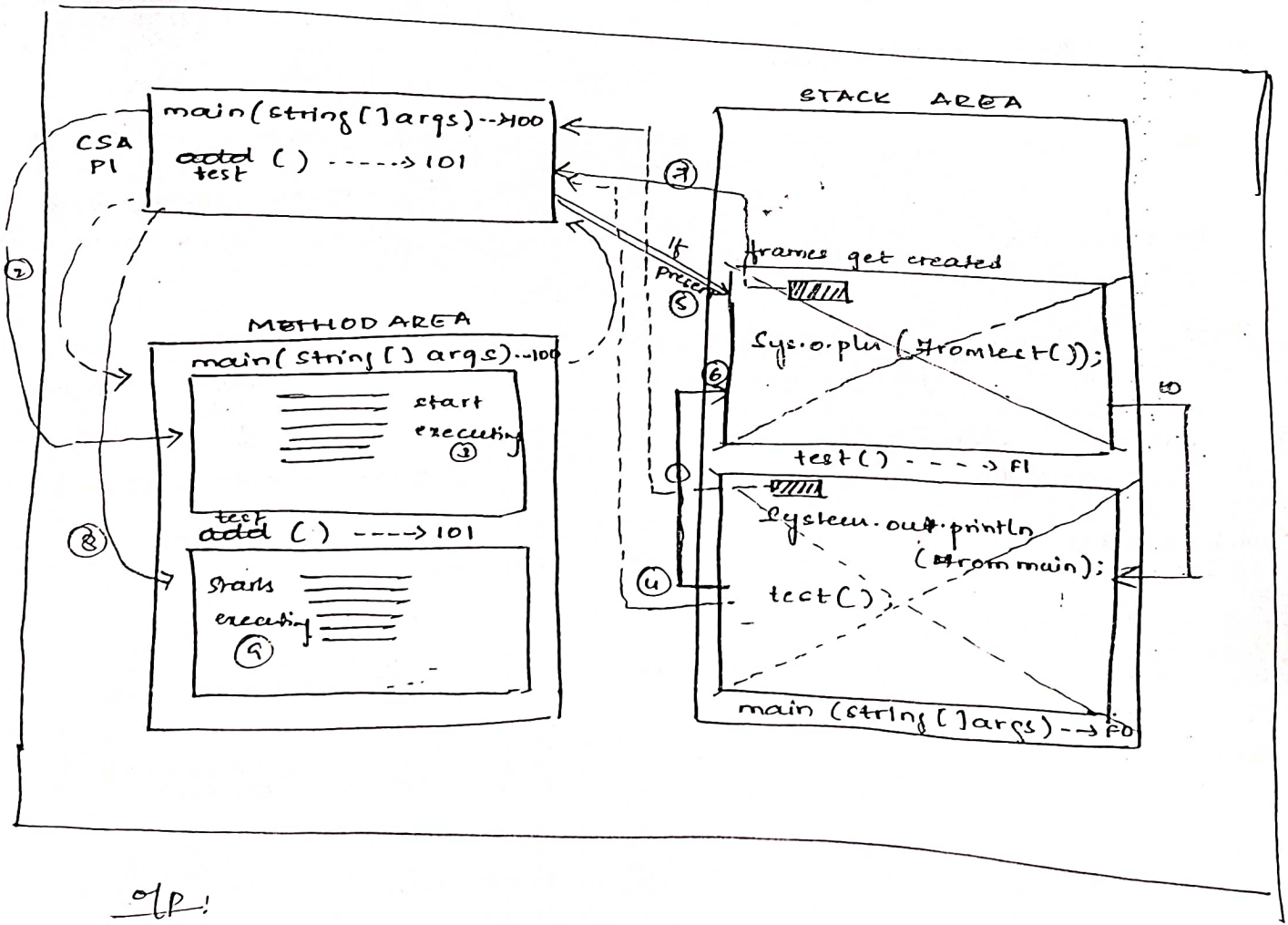
* A block which belongs to a static member is known as **STATIC CONTEXT.**

* For executing a static content we will have a frame getting created inside stack area. ~~class static area~~ which is known as **STATIC FRAME.**

* Every static frame will have a built-in reference pointing to its class-static area.

Ex:

```
class P1
{
~~public~~ static void test()
 {
  System.out.println("From test()");
 }
public static void main(String[] args)
 {
 System.out.println("From main");
  test();
 }
}
```



O/P:

From main

From test

* The static members of the class can be used within the static context in following ways.

1) Directly with the help of member name or Member signature.

2) With the help of class using access operator (.)

ACCESS OPERATOR (.) :-

1) It is a binary Operator.

2) It is used to access a member present a particular location / address.

SYNTAX :-

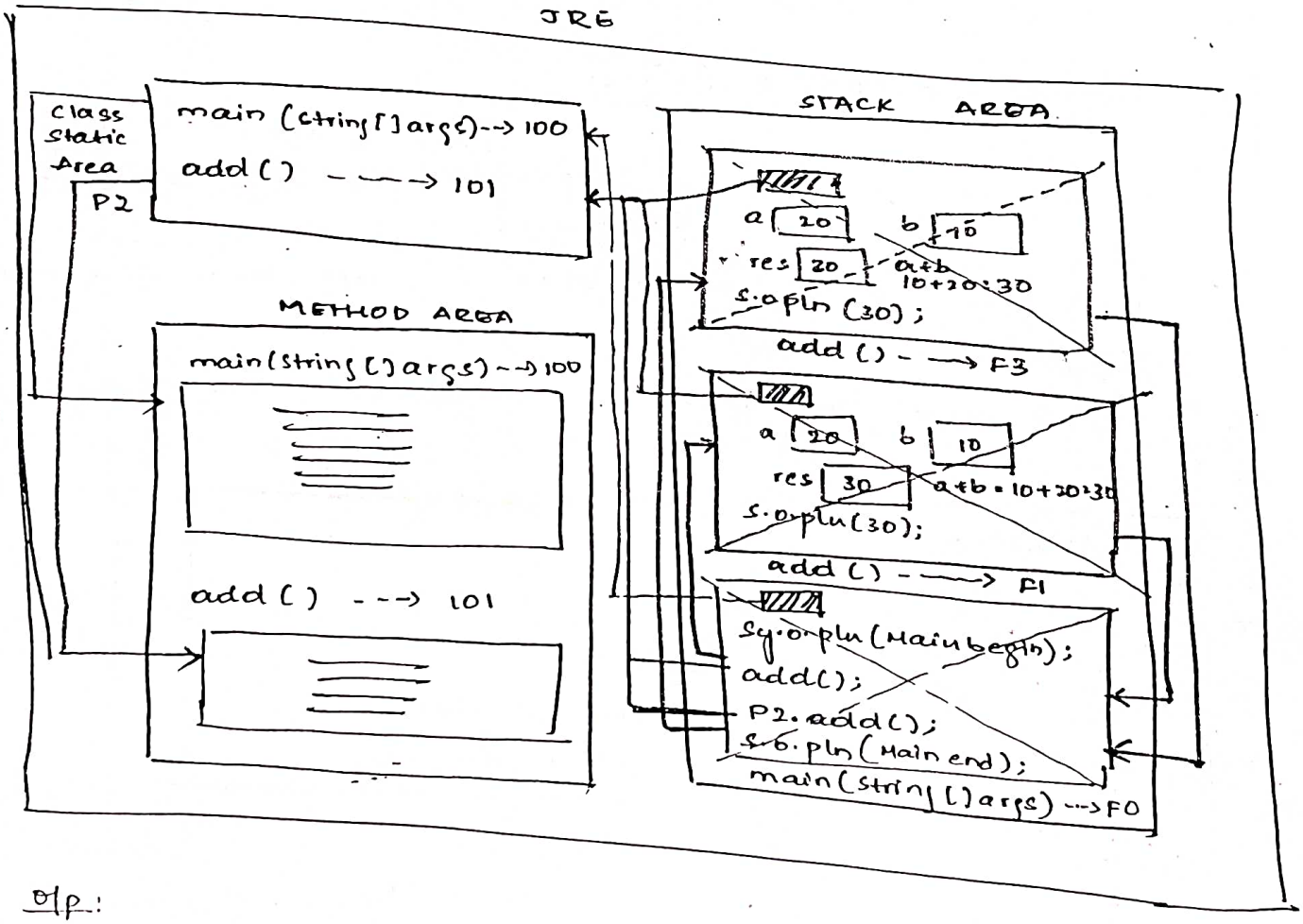
LHS . RHS.

LHS ⇙ address/reference

RHS ⇘ Member present in that address

Eg :
```
class P2
{
public static void add ()
{
int a = 20;
int b = 10;
int res = a+b;
```

```
System.out.println(res);
}
public static void main(String[] args)
{
System.out.println("Main Begin");
add();
P2.add();
System.out.println("Main end");
}
}
```



Op:

Main begin

30

30

Main end

08/02/2022

Q. Create an application which is used to perform addition & subtraction of two numbers. (Design static methods for each task).

```
class Addsub
{
 public static int add (int a, int b)
 {
  return a+b;
 }
 public static int sub (int a, int b)
 {
  return a-b;
 }
 public static void main (String [] args)
 {
  System.out.println ("The addition of two numbers is:");
  System.out.println (add (10, 20));
  System.out.println ("The subtraction of two numbers is:");
  System.out.println (sub (20, 10));
 }
}
```

Q How to use a static member of one class inside static context of another class.

We can use static members inside another class with the help of class name along with access operator.

NOTE :

▶ INITIAL CLASS:

The class which is given as input to java command is known as Initial class.

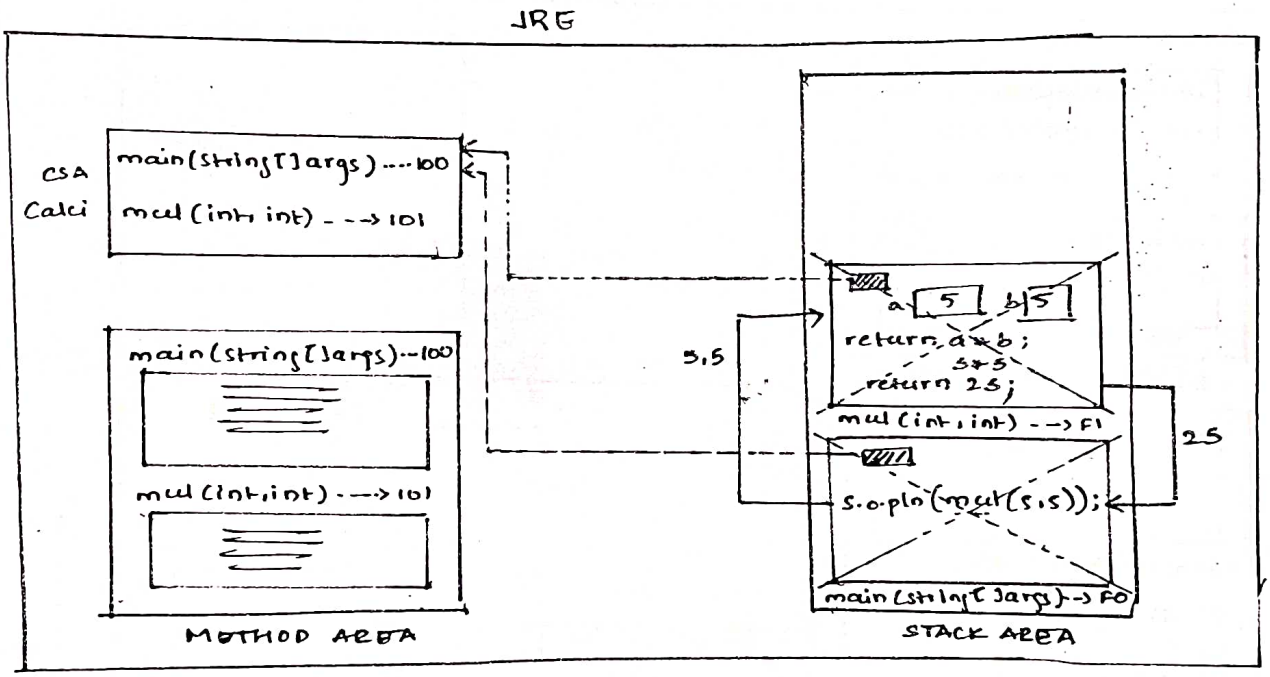
▶ DEPENDANT CLASS:

A class used inside another class is known as dependant class.

Ex:
```
class Calculator
{
public static int add (int a, int b)
{
return a+b;
}
public static int sub (int a, int b)
{
return a-b,
}
}
```

Ex:

```
class Calci
{
public static void main(String[] args)
 {
 System.out.println(mul(5,5));
 }
public static int mul(int a, int b)
 {
 return a*b;
 }
}
```

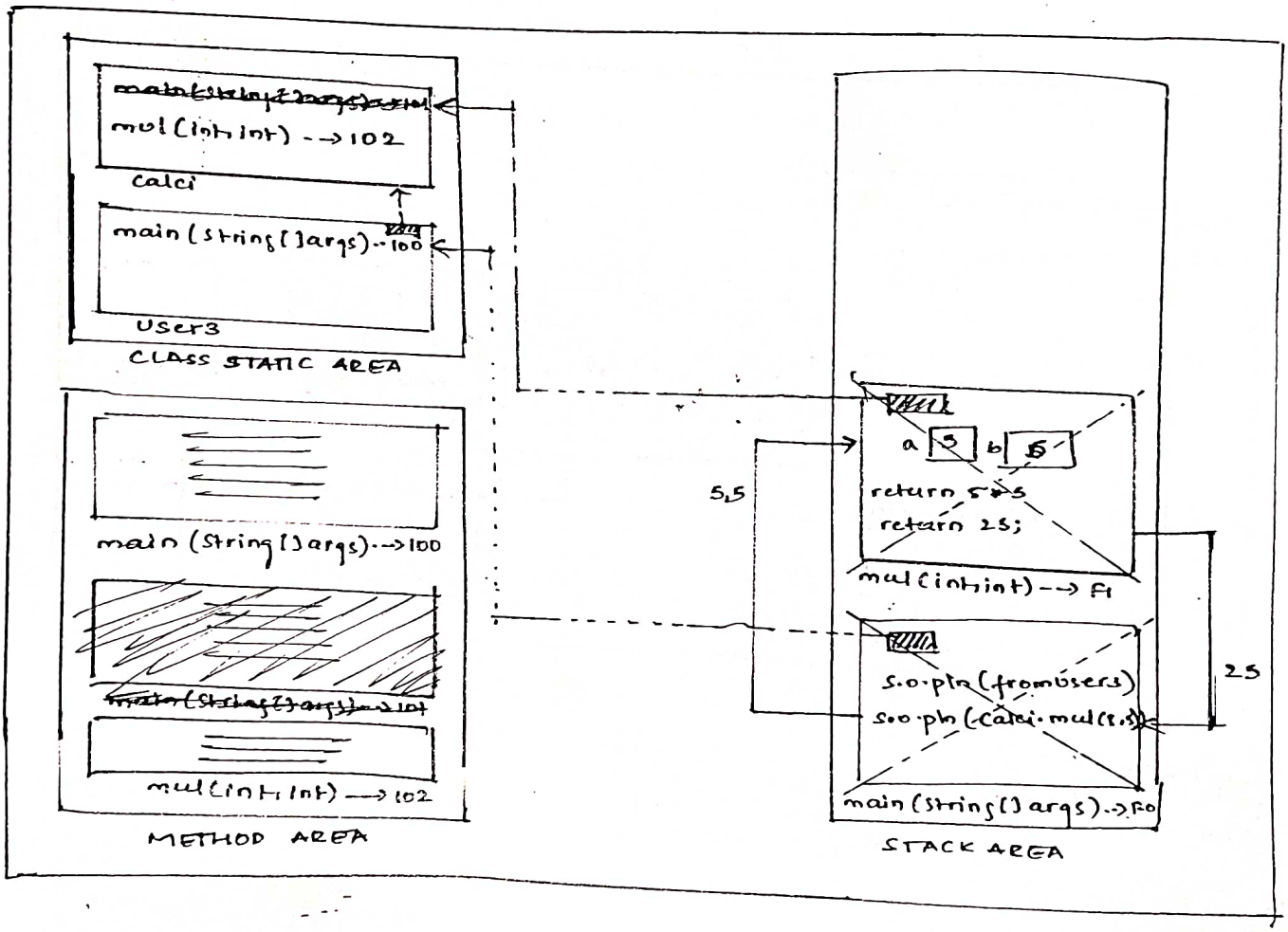


o/p :

25

Q. class User3

```
{
public static void main (String [] args)
{
System.out.println ("from User3");
System.out.println(Calci.mul(5,5));
}
}
```



o/p:

from user 3

25

09/02/2022

STATIC VARIABLE

A Global variable which is prefixed with static modifier is known as static variable

NOTE: A local variable cannot be made static

Points to Remember :-

1) A static variable gets its memory allocated inside class static area.

2) Since all static variables will be present inside class static area, they are also known as CLASS VARIABLES.

3) A static variable will be assigned with default values.

Ex:
```
class P2
{
static int a;
public static void main (String [] args)
{
  System.out.println(a);
}
}
```

O/P:
0

4) A static variable can be used within static context of the same class in two ways: They are:

i) Directly with the help of name of a variable

ii) With the help of class name.

Ex:
```
class P2
{
static int a;
public static void main (String [] args)
{
System.out.println (a);
System.out.println (P2.a);
}
}
```

O/P:
```
0
0
```

5) A static variable of one class can be accessed inside another class with the help of class name

Ex:
```
class P2
{
static int a;
}

class P5
{
static double a;
}

class P6
{
public static void main (String [] args)
{
System.out.println (P2.a);
```

System.out.println(P5.a);

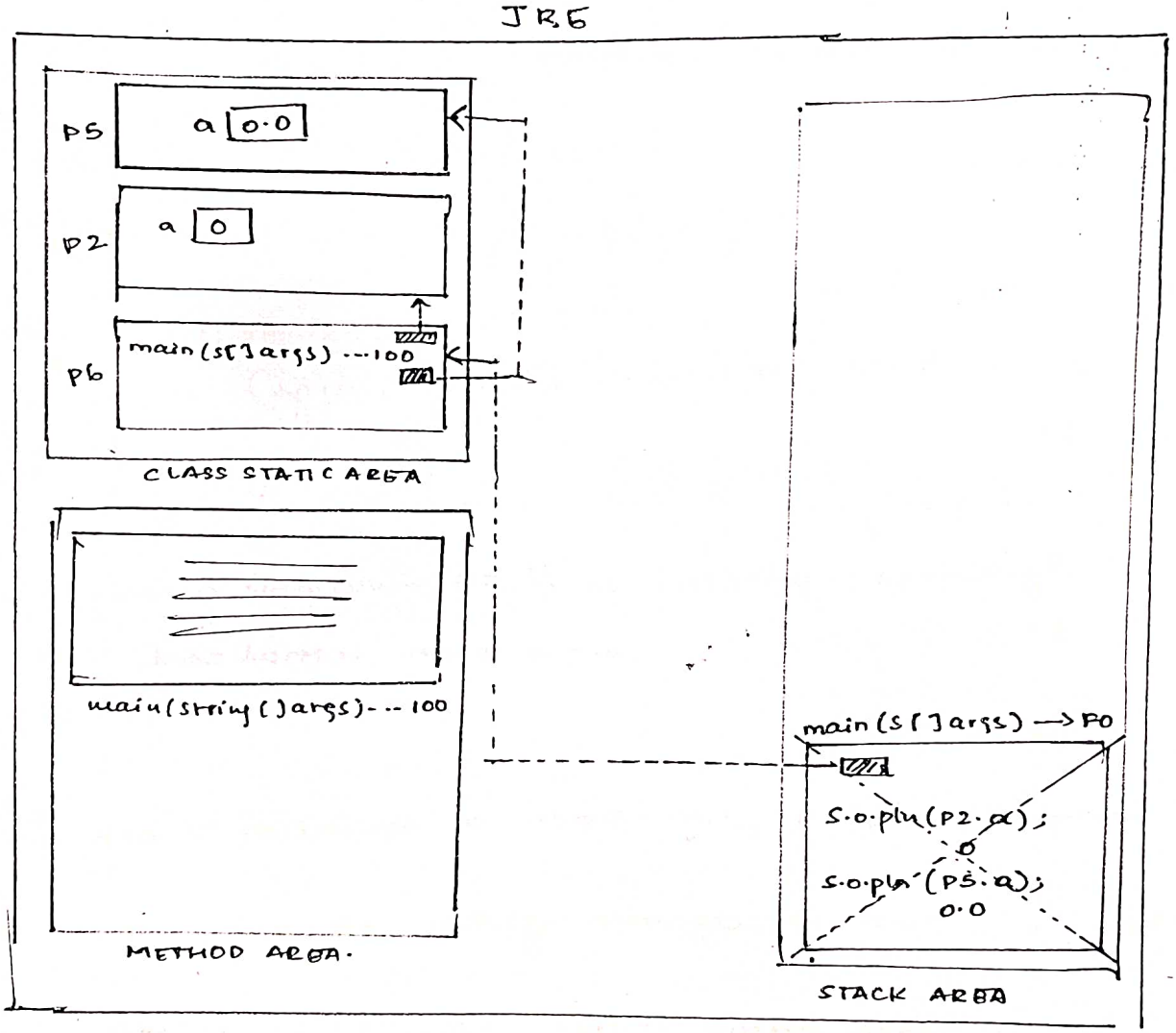
}

}

o/p :

0

0.0



o/p:

0

0.0

6) In java we can have local variable as well as global variable with the same name.

7) In such situation, the highest priority will be given to local variables.

Ex. class P7
{
static int a;
public static void main (String [] args)
{
int a;
System.out.println(a); // CTE - variable a might not have been initialized
}
}

8) ~~In the abo~~ If we have a situation like p1.no then we can access the global variable with the help of class name itself. We cannot access it directly.

Ex: class P8
{
static int a;
public static void main (String [] args)
{
int a = 30;
System.out.println(a);
System.out.println(P8.a);
}

O/P:
30
0

9) Global Variable can be used inside all the members of the class whereas the local variables can be used only inside the block where it is declared.

Ex: class P10
{
static int a;
public static void main (String[] args)
{
System.out.println(a);
Senorita();
}
public static void Senorita (~~String[]~~)
{
System.out.println(a);
}
}

~~O/P~~:
0
0

STATIC INITIALISERS

Initialisers are the members of the class which are used for initialisation purpose.

In java, we have two types of Initialisers:

1. Static declare & initialisation statement
2. static block.

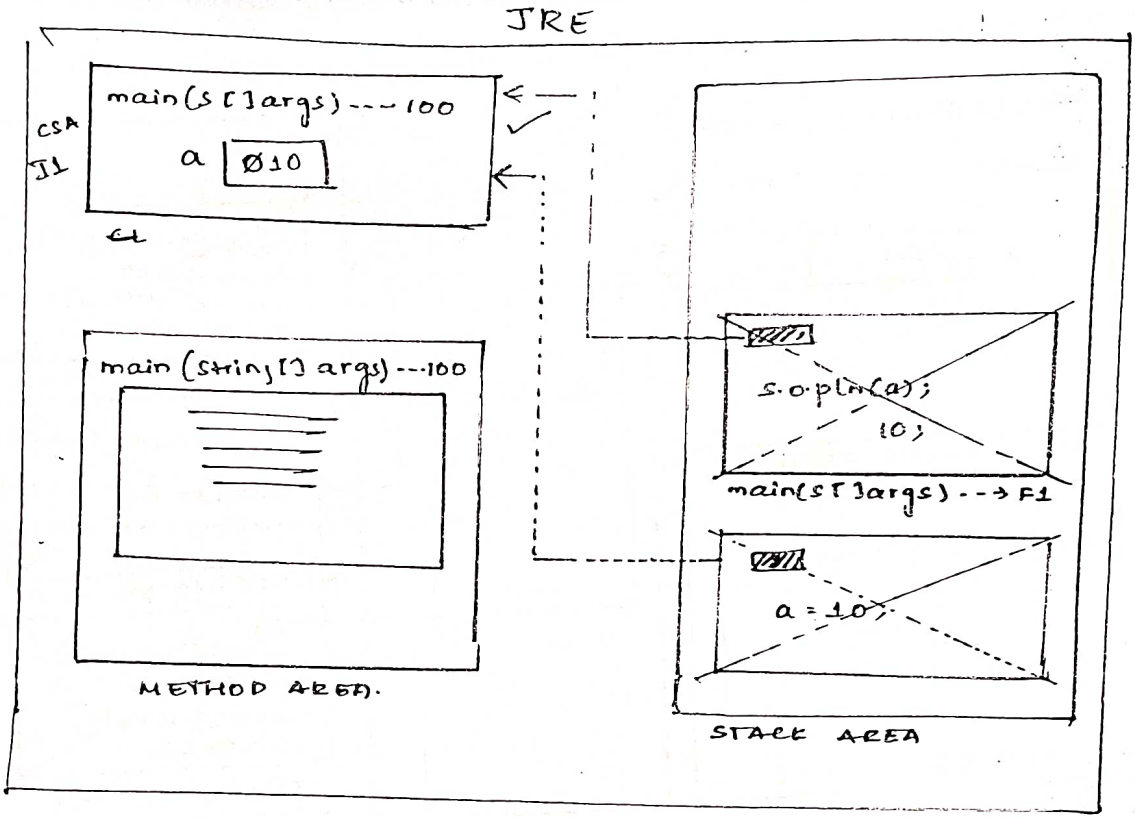
▶ STATIC DECLARE & INITIALISATION STATEMENT:

```
static datatype identifier = value/ Expression/ ;
                                    statement
```

1) The static initialisers gets executed during loading process of a class.

2) The static initialisers gets executed only once for class loading.

Ex:
```
class I1
{
static int a = 10;
public static void main (String[] args)
{
System.out.println(a);
}
}
```

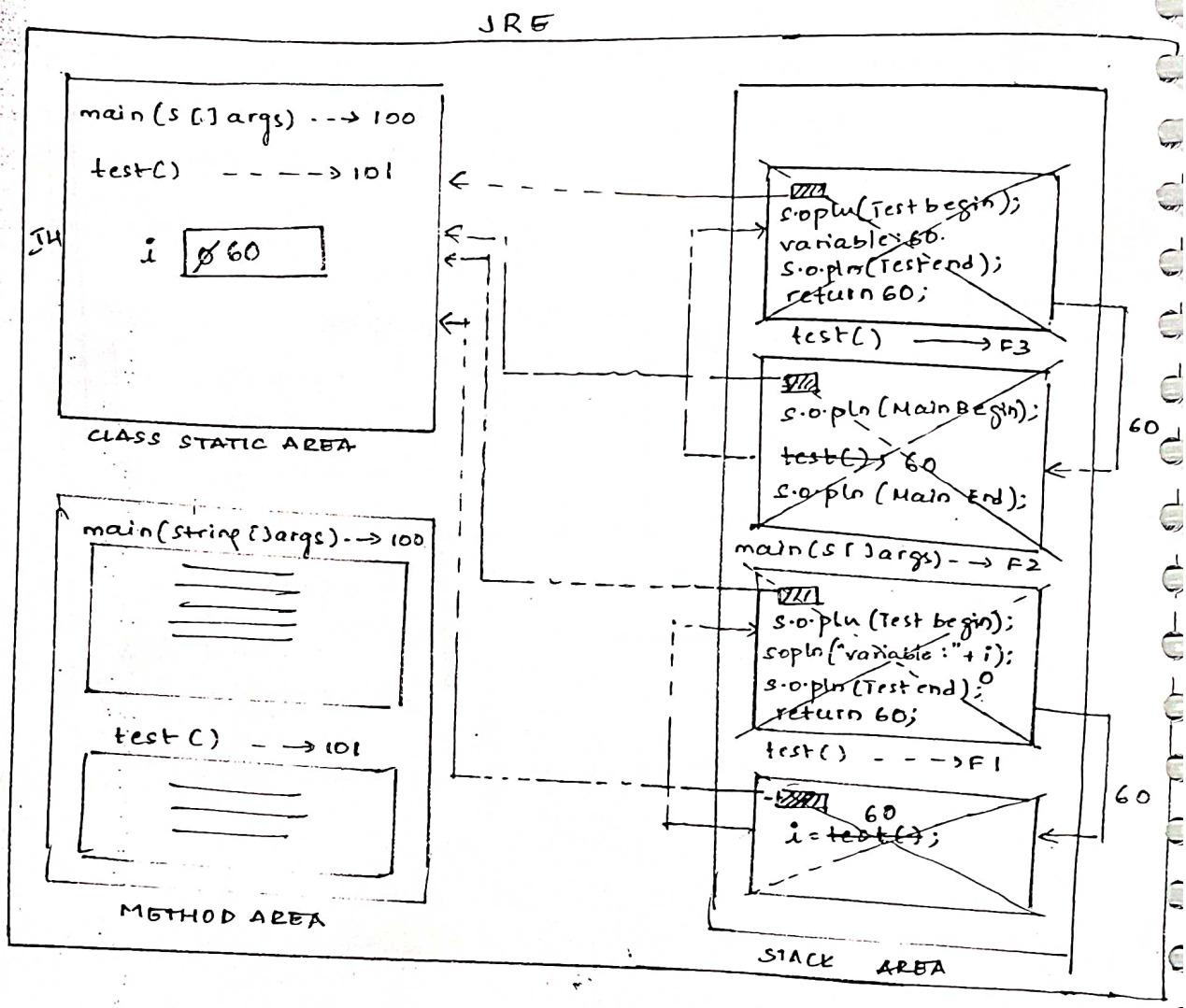


o/p:

10

Ex: class I4

```
{
public static int test()
{
System.out.println("Test Begin");
System.out.println("variable :" + i);
System.out.println("Test end");
return 60;
}
static int i = test();
public static void main(String[] args)
{
System.out.println("Main Begin");
test();
System.out.println("Main End");
}
}
```



o/p:

Test Begin

variable : 0

Test end:

Main Begin

Test Begin

Variable : 60

Test end

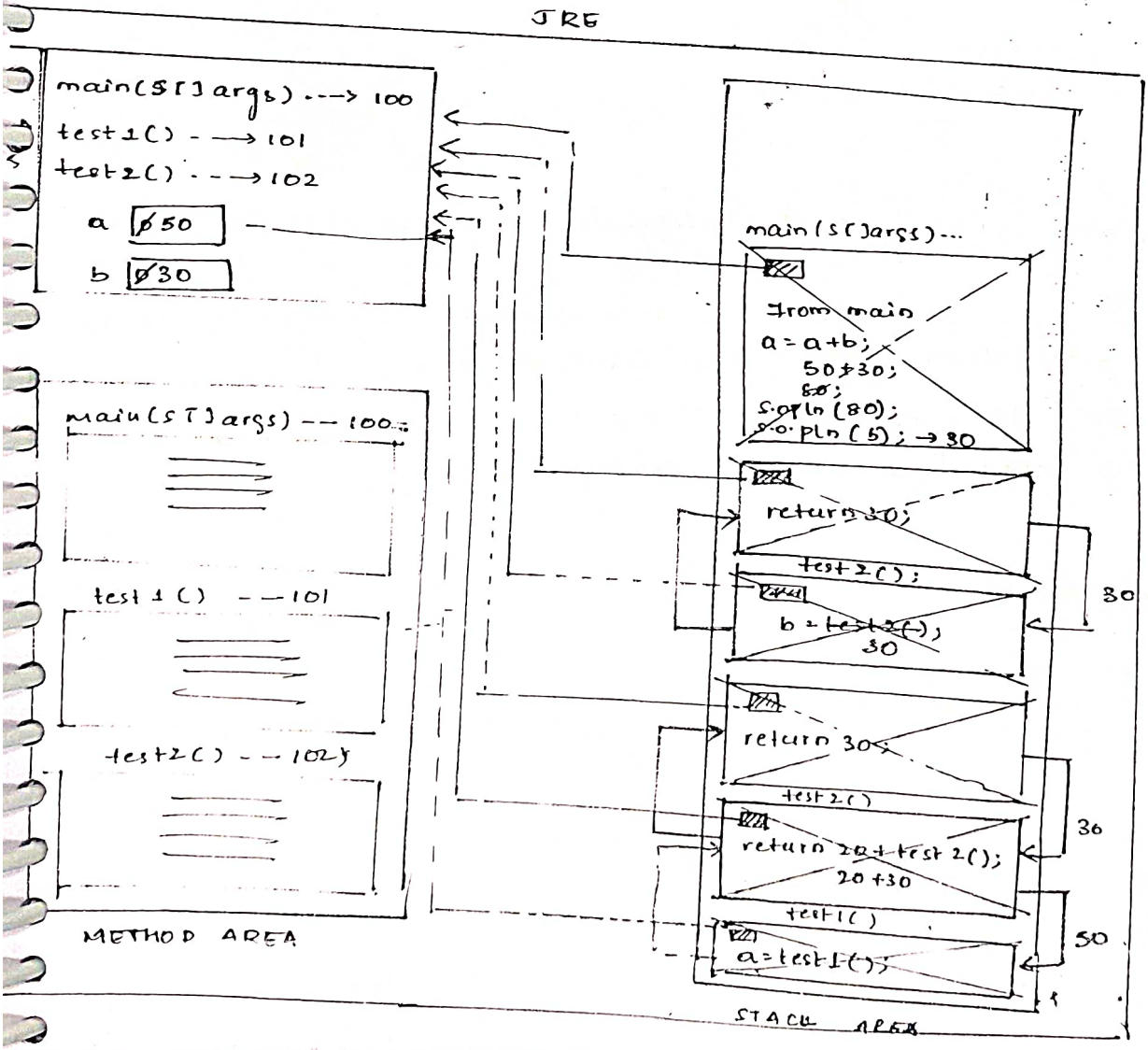
Main End

Ex :- class Ex5

```
{
    static int a = test1();
public static int test1()
{
return 20 + test2();
}
public static void main(String[] args)
{
System.out.println("From main");
a = a + b;
System.out.println(a);
System.out.println(b);
}
public static int test2()
{
return 30;
}
static int b = test2();
}
```

o/p:

From main
80
30



STATIC BLOCK :-

SYNTAX:

```
static
{

    //statements;

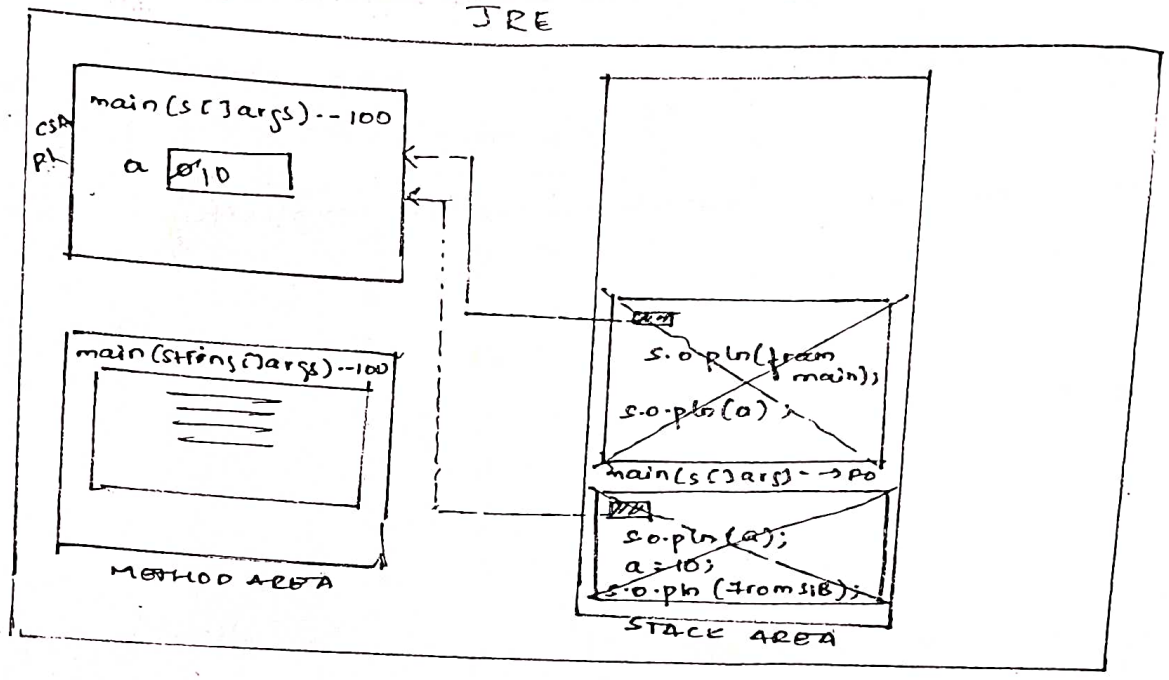
}
```

An anonymous block which is prefixed with static modifier is known as static block.

- It is also static Initialiser.

Ex : class P1

```
{
static int a;
static
 {
  S.o.pln (a);
  a = 10;
  S.o.pln("From Static Initialiser Block");
 }
public static void main (String[] args)
{
S.o.pln("From main");
S.o.pln (a);
}
}
```



o/p:

0

from SIB

from main

10

10/02/2022

NOTE:

The static block gets executed during the loading process of class (ie before the execution of main method).

The static block gets executed only once for the loading process of a class.

CHARACTERISTIC OF A STATIC BLOCK:-

1) doesn't have a name

2) doesn't have a return type

3) doesn't have a formal argument

4) It cannot accept any i/p's from the user.

5) The programmer cannot call it explicitly

Ex:

```
class A
{
 static
 {
 System.out.println ("SIB-1");
 }
public static void main (String[] args)
 {
 System.out.println ("From main");
 }
}
```

o/p:

SIB-1

From main.

d).

NOTE :

Inside a class we can have more than one static block & they get executed in Top to Bottom order.

Ex:

```
class B
{
 static
 {
 s o pln ("SIB 1"),
 }
```

```
public static void main(String[] args)
{
System.out.println("from main");
}
static
{
System.out.println("SIB-2");
}
}
```

O/P:

SIB-1

SIB-2

from main.

DIFFERENTIATE BETWEEN STATIC BLOCK & STATIC METHOD.

STATIC BLOCK	STATIC METHOD.
Static Blocks will not be having any name	Static Methods will have a name
It doesn't have a return type	It must have a return type
cannot accept any inputs or any arguments	Can pass actual arguements.
Static blocks gets executed during loading process of class	Static methods gets executed only when it is called.

5) It gets executed only once for the loading process of a class	It can gets executed multiple times based on the user calling it.

LOADING PROCESS OF A CLASS:

1) A class Static Area will be created for given class (initial class).

2) All the methods (static as well as non-static) will be loaded to Method Area.

3) If a static Methods are present, its reference will be stored (Method signature + address) in class static area

4) If a static variable is present, it will get its mem allocated inside class static area with default value

5) If static initialisers are present, they get executed directly.

6) The loading process is said to be complete

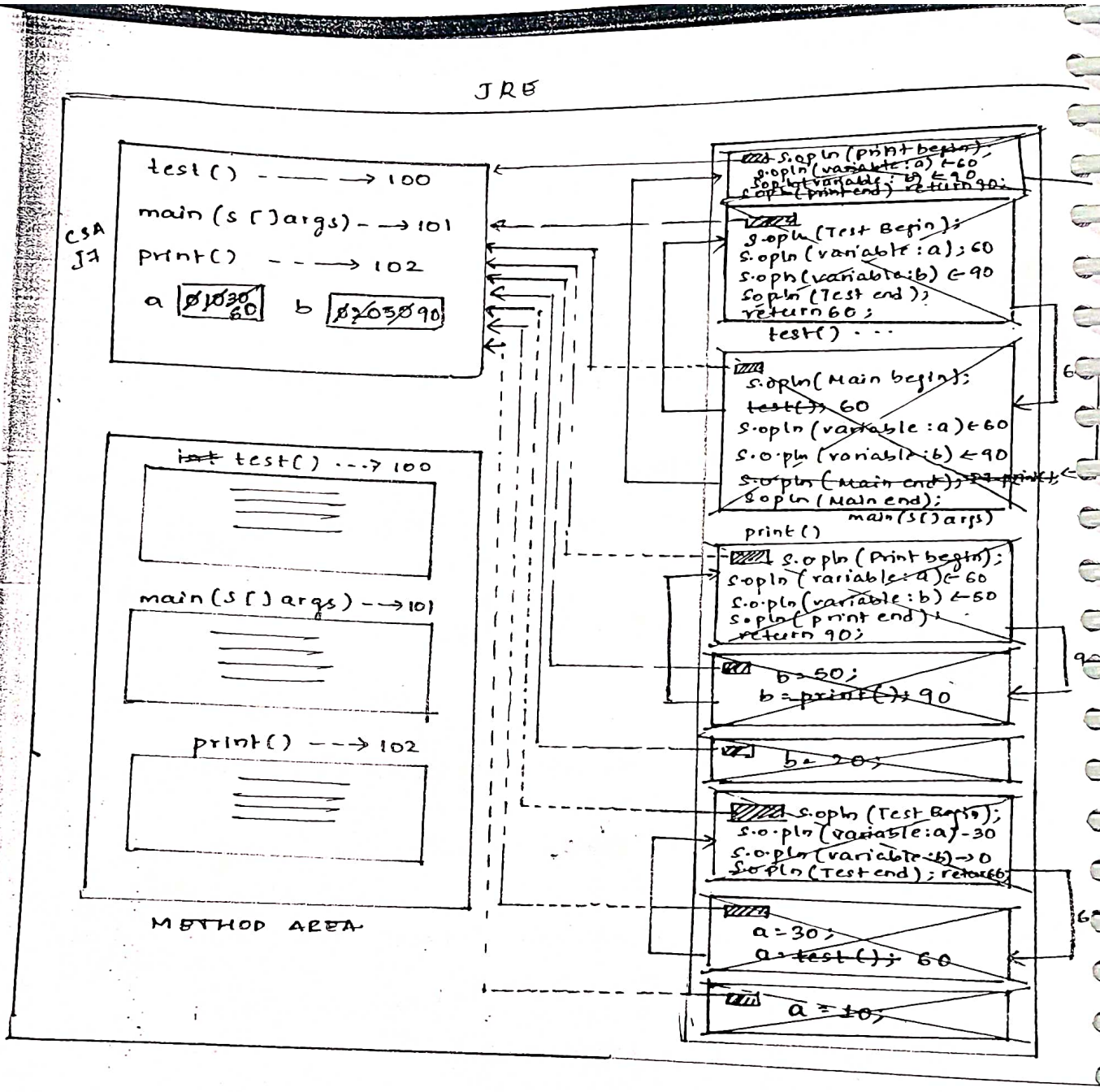
Ex:

```
class JT
{
static int a = 10;
static
{
 a = 30;
 a = test();
}
public static int test()
{
System.out.println("Test Begin");
System.out.println("variable :" + a);
System.out.println("variable: " + b);
System.out.println("Test end");
return 60;
}
public static void main(String[] args)
{
System.out.println("Main Begin");
test();
System.out.println("variable: " + a);
System.out.println("variable: " + b);
JT.print();
System.out.println("Main end");
}
public static int print()
{
System.out.println("Print Begin");
System.out.println("variable: " + a);
System.out.println("variable: " + b);
System.out.println("print end");
return 90;
}
static int b = 20;
static
{
 b = 50;
} b = print();
```

CSA
JT

o/p

Test
var
var
Tes
Pri
va
va
Pri
M
Te
va
va
Te



o/p:

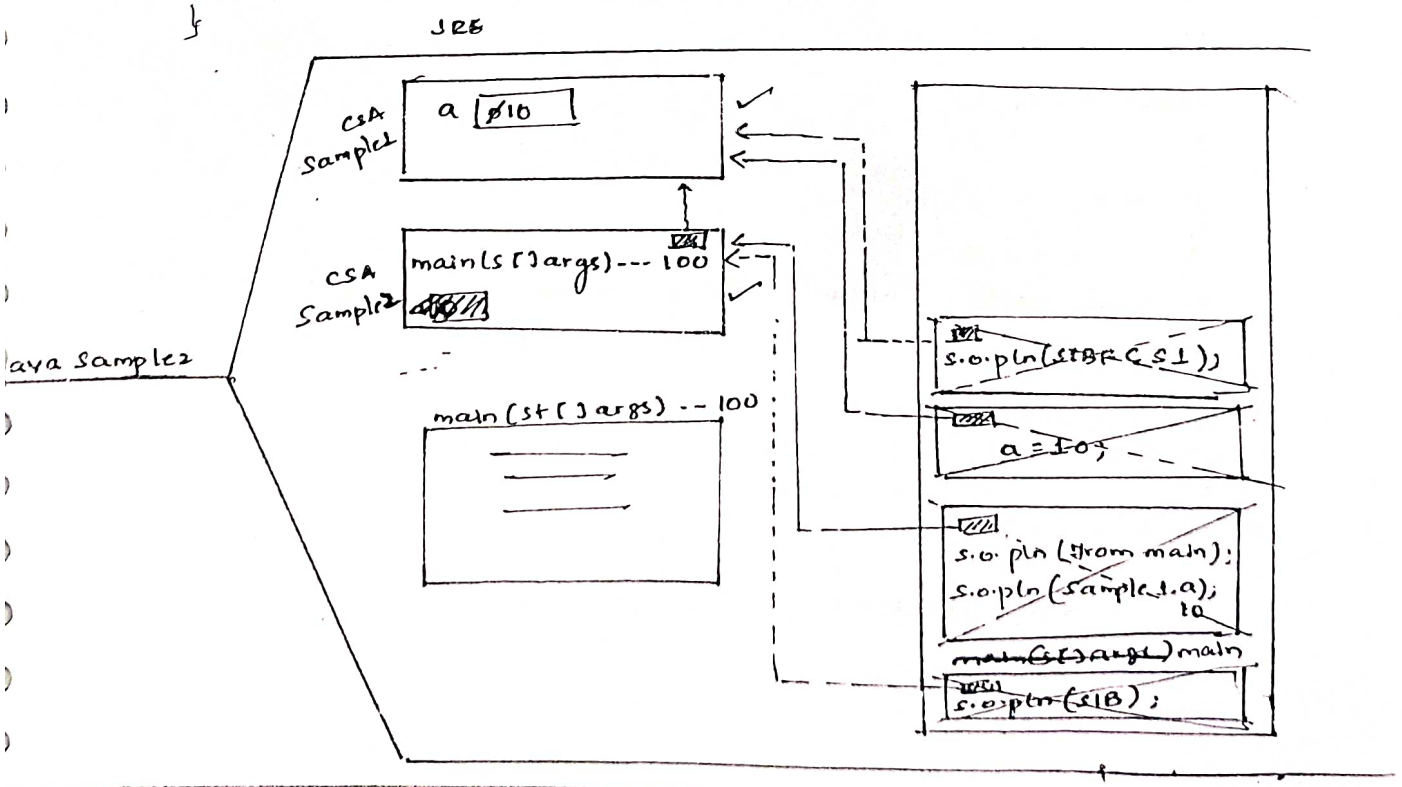
Test Begin
variable: 30
variable: 0
Test end.
Print Begin.
variable: 60
variable: 50
print end
Main begin
Test begin
variable: 60
variable: 90
Test end
variable: 60
variable: 90

print begin
variable: 60
variable: 90
print end
Main end

Ex.

```
class Sample1
{
static int a = 10;
static
{
System.out.println ("SIB from class sample 1");
}
public static void main (String[]args)
{
System.out.println ("From main");
System.out.println(a);
}
}
```

```
class Sample 2
{
static
{
System.out.println ("SIB of class sample 2");
}
public static void main (String[]args)
{
System.out.println ("From main");
System.out.println (Sample1.a);
}
}
```



HP:

SIB of class Sample2

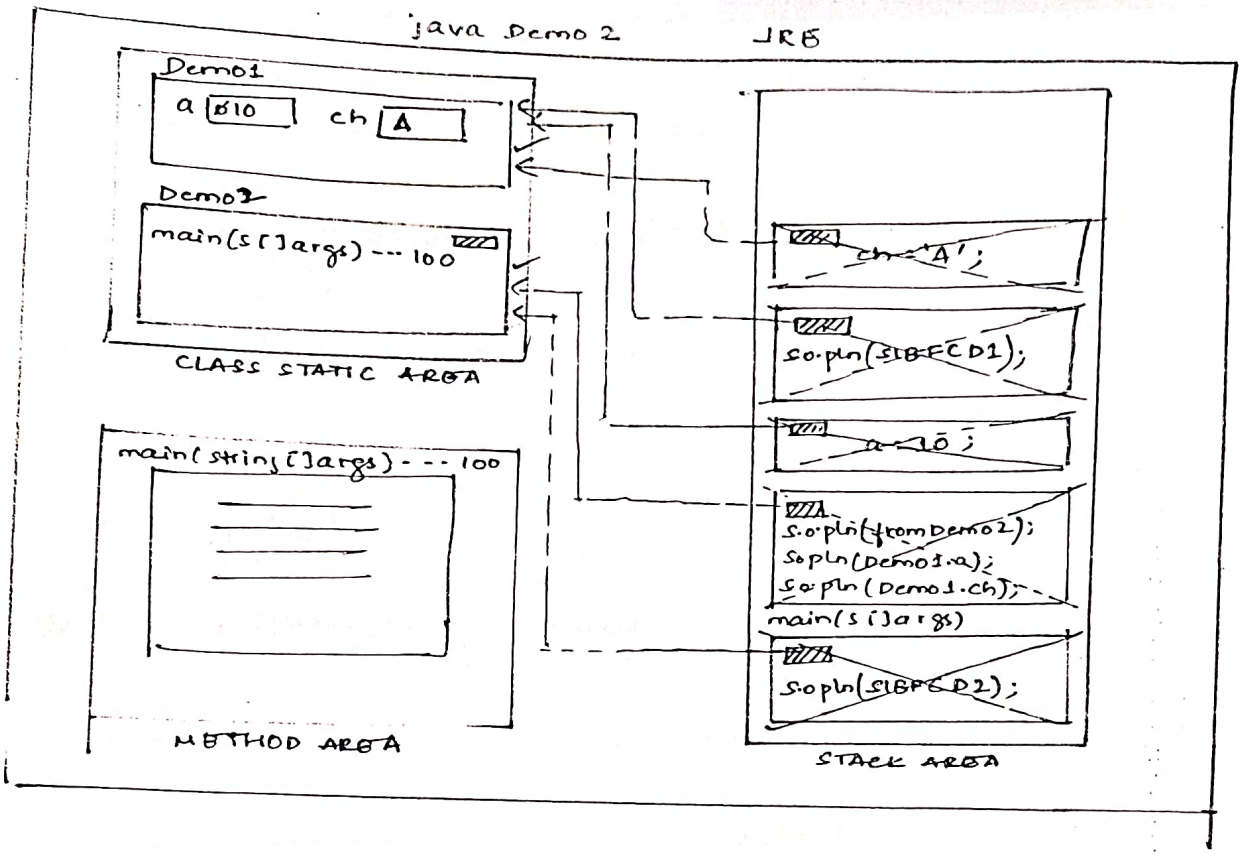
from main:

SIB from class sample1

10

STATIC ~~CODE~~ ~~INITIALISE~~ BLOCKS GETS EXECUTED ONLY ONCE DURING LOADING PROCESS:

```
Ex: class Demo1
    {
    static int a=10;
    static
     {
      System.out.println("SIB from class Demo1");
     }
    public static void main(String[] args)
     {
      System.out.println("From Demo1");
      System.out.println(a);
      System.out.println(ch);
     }
     static char ch='A';
    }
    class Demo2
    {
     static
     {
     System.out.println("SIB from class Demo2");
     }
     public static void main(String[] args)
     {
     System.out.println("From Demo2");
     System.out.println(Demo1.a);
     System.out.println(Demo1.ch);
     }
    }
```



O/p:

SIB from class Demo2

From Demo2

SIB from class Demo1

10

A